

Calendar Application Test Plan

Authors: Ben Jones & Team

Note to the grader: Given the extent of tests made, the documentation was rather lengthy. AI was used in the formatting and paraphrasing of the tests — please note the output of the model is probabilistic and subject to potential errors; all code was human-reviewed for accuracy. The choice to write the document in Markdown prior to pasting it into the document was intentional, as it is easier to review.

The Test Plan documents automated tests and manual tests that should give stakeholders confidence that the application has implemented a set of requirements.

Unit Testing

The unit tests for this application focus on verifying that core calendar view components render correctly when provided with expected props. Specifically, the `DayBody`, `MonthBody`, `WeekBody`, and many more components are tested using React Testing Library to ensure they mount successfully and are present in the DOM. These tests validate basic component integrity by checking for elements via `data-testid` attributes.

The current level of test coverage is foundational but limited in scope. While rendering tests confirm that components do not crash and appear correctly, they do not yet validate dynamic behavior such as event handling, conditional rendering, or data-driven updates. Components like `DayBody`, `MonthBody`, and `WeekBody` have reasonable baseline tests, but they would benefit from additional coverage that tests user interactions (e.g., clicking dates, opening modals) and edge cases (e.g., empty states, invalid dates).

Components that should have more unit tests include:

- Modal interactions (`openModal`)
 - Event/task rendering within each calendar view
 - State-dependent UI
-

Use Case Testing

A use case test includes test situations that test all the steps in a use case, to verify the system implementation of the use case.

Use case tests include automated tests written in code, and manual tests written in this document.

Automated Use Case Tests

There are currently no fully implemented automated use case tests that simulate complete user workflows (e.g., navigating between views or creating events). The existing tests instead validate individual UI components in isolation. These correspond loosely to early-stage use cases such as:

- **UCD-0001:** View Day Calendar
- **UCD-0002:** View Month Calendar
- **UCD-0003:** View Week Calendar

Future work should expand these into full workflow tests (e.g., switching views, interacting with dates, triggering modals).

Manual Use Case Tests

UCD-0001 — View Day Calendar

Field	Value
Name/Title	View Day Calendar
ID	0001
Description	The user is able to view the calendar in day format.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to see a single day's schedule.

Related Use Cases	None
Invariant	—
Precondition	The user has opened the calendar application.
Success Postcondition	The day view is displayed with the correct date.

User Steps	System Response
1. The user opens the calendar application.	1. The system renders the Day view interface.
2. The tester ensures the date is set to June 15, 2024.	2. The system displays the correct day layout.

Postcondition Testing Steps:

- Verify that the day view is visible on the screen.
- Verify that elements with `daybody` test identifier are present.

UCD-0002 — View Month Calendar

Field	Value
Name/Title	View Month Calendar
ID	0002
Description	The user is able to view the calendar in month format.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to see a full month overview.
Related Use Cases	None
Invariant	—

Precondition	The user has opened the calendar application.
Success Postcondition	The month view is displayed correctly.

User Steps	System Response
1. The user opens the calendar application.	1. The system renders the Month view interface.
2. The tester sets the date to June 15, 2024.	—

Postcondition Testing Steps:

- Verify that the month view is visible.
- Verify that elements with `monthbody` test identifier are present.

UCD-0003 — View Week Calendar

Field	Value
Name/Title	View Week Calendar
ID	0003
Description	The user is able to view the calendar in week format.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to see a weekly schedule.
Related Use Cases	None
Invariant	—
Precondition	The user has opened the calendar application.
Success Postcondition	The week view is displayed correctly.

User Steps	System Response
1. The user opens the calendar application.	1. The system renders the Week view interface.
2. The tester sets the date to June 15, 2024.	—

Postcondition Testing Steps:

- Verify that the week view is visible.
- Verify that elements with `weekbody` test identifier are present.

UCD-0004 — Create a Task

Field	Value
Name/Title	Create a Task
ID	0004
Description	The user is able to create a task to add to their calendar/to-do list/kanban board.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to create tasks and set due dates for them.
Related Use Cases	Included by: 0005 Add Subtasks to Task
Invariant	—
Precondition	The user has logged in.
Success Postcondition	A task is created and the details are viewable in the task list, with the correct field values.

--	--

User Steps	System Response
1. The user clicks the "Create a Task" button on the page.	2. The system displays the task page with the fields to fill out (task name, description, due date).
3. The user fills in the details as required (task name = "Test task 1", description = "Test task description 1", due date = "January 5, 2022") and clicks the "Add Task" button.	4. The system adds the task to the calendar/to-do list and adds the details to the database.

Postcondition Testing Steps:

- Verify that the created task appears in the task list.
- Verify that the task name, description, and due date match the entered values.

UCD-0005 — View Calendar Day Cells

Field	Value
Name/Title	View Calendar Day Cells
ID	0005
Description	The user is able to view individual day cells across different calendar views (day, week, month).
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to visually interpret dates within the calendar interface.
Related Use Cases	Used by: 0001 View Day Calendar, 0002 View Month Calendar, 0003 View Week Calendar
Invariant	Each calendar view must render a valid day representation.
Precondition	The user has opened the calendar application.

Success Postcondition	Day cells are displayed correctly in all calendar views.
-----------------------	--

User Steps	System Response
1. Open the calendar application.	1. Render <code>DayDay</code> component in Day view.
2. Navigate to Day view, Month view, and Week view.	2. Render <code>MonthDay</code> component in Month view.
3. Observe rendered day cells for June 15, 2024.	3. Render <code>WeekDay</code> component in Week view.

Postcondition Testing Steps:

- Verify `dayday` element exists in Day view.
- Verify `monthday` element exists in Month view.
- Verify `weekday` element exists in Week view.

Evidence Mapping (Jest → Behavior):

Component	Test Validation
<code>DayDay</code>	Present in DOM via <code>getByTestId("dayday")</code>
<code>MonthDay</code>	Present in DOM via <code>getByTestId("monthday")</code>
<code>WeekDay</code>	Present in DOM via <code>getByTestId("weekday")</code>

Architecture Context — Layered Test Coverage:

Layer	Status
Fields	✓
Bodies	✓
Interactions	✓

Gap Identified: Missing cross-component workflows (e.g., click day → modal → create event → appears in view).

UCD-0007 — Manage Tags

Field	Value
Name/Title	Manage Tags
ID	0007
Description	The user can create, view, sort, and retrieve tags used for organizing tasks and events.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to organize items using tags with names and colors.
Related Use Cases	Supports: Task Management, Calendar Organization
Invariant	Tags must contain a valid <code>id</code> , <code>name</code> , and <code>color</code> .
Precondition	The user is interacting with the tag system.
Success Postcondition	Tags are correctly created, sorted, and retrievable.

User Steps	System Response
1. Create three tags: ID "2" Name "Work" Color "#FF0000"; ID "1" Name "Personal" Color "#00FF00"; ID "3" Name "Urgent" Color "#0000FF".	1. Store tags with correct properties.
2. Sort tags alphabetically by name.	2. Sort tags into order: Personal → Urgent → Work.
3. Retrieve tag with ID "2".	3. Return tag with Name "Work" and Color "#FF0000".

Postcondition Testing Steps:

- Verify tag object fields match assigned values.
- Verify sorted order is correct (Personal → Urgent → Work).
- Verify retrieved tag matches ID "2".

Evidence Mapping (Jest → Behavior):

Operation	Test Validation
Tag Creation	Properties match expected values
Tag Sorting	Ordered via <code>localeCompare</code>
Tag Retrieval	Found via <code>find(tag.id === target)</code>

Architecture Context — Layered Test Coverage:

Layer	Status
Data Models	✓
Fields	✓
Bodies	✓
Interactions	✓
Pages	✓

Gaps Identified:

- No persistence layer validation (DB/API).
- No UI integration (tags displayed in tasks/calendar).
- No mutation workflows (edit/delete tags).

UCD-0008 — View Task Item

Field	Value
Name/Title	View Task Item

ID	0008
Description	The user can view an individual task item including its name and completion checkbox.
System Under Design	BetterDays
Primary Actor	User
Participants	None
Goal	To allow users to see task details and completion status.
Related Use Cases	Supports: Task Management
Invariant	A task item must display a checkbox and task name.
Precondition	The user is on a page displaying tasks.
Success Postcondition	Task item is visible with checkbox and name displayed.

User Steps	System Response
1. Navigate to the Task page.	1. Render <code>TaskItem</code> component.
2. Observe a task item in the list.	2. Display checkbox for task completion.
—	3. Display task name.

Postcondition Testing Steps:

- Verify `taskitem` element exists.
- Verify `checkbox` element exists.
- Verify `taskname` element exists.

Evidence Mapping (Jest → Behavior):

Component	Test Validation
<code>TaskItem</code>	Present via <code>getByTestId("taskitem")</code>

Checkbox	Present via <code>getById("checkbox")</code>
Task Name	Present via <code>getById("taskname")</code>

Architecture Context — Layered Test Coverage:

Layer	Status
Data Models	✓
Fields	✓
Bodies	✓
Interactions	✓
Pages	✓
Task Components	✓

Gaps Identified:

- No interaction testing (checkbox toggle).
- No modal interaction (`openModal`).
- No task state validation (completed vs. incomplete).

Acceptance Testing

For this project, use case tests were executed manually and results are documented below.

Results Summary

Use Case	Result
UCD-0001 View Day Calendar	✓ Passing
UCD-0002 View Month Calendar	✓ Passing
UCD-0003 View Week Calendar	✓ Passing
UCD-0004 Create a Task	✓ Passing